Generation Paradigm (Score Match)

Motivation

Score Match is a new way to make generation paradigm different from diffusion

Goal: to sample a new img from the distribution of data set p_{data}

a common strategy: sample from an easy distribution and recover the img that we want to sample towards the end

details:

1. we sample a x which isn't from p_{data} but from a simple distribution, like Gaussian noise
2. the key is we wanna to find a way that make us go to somewhere that is hard to sample like p_{data} from somewhere that is easy to sample like x
3. suppose we have the probabilty density function $p(x)$ that could give us the probabilty density value of x we sampled
4. easy to say that we can use gradient to get a x that make $p(x)$ have a higher value

Some prerequisite:

Gradient of log of $p(x)$

we won't directly use $\nabla p(x)$ because $\int p(x)dx = 1$ and it can be seen as $p(x) = fracf(x)Z$, where the Z is intractable so $\nabla p(x) = \nabla \frac{f(x)}{Z}$ is still intractable

as well, $p(x)$ is easy to meet numerical issues because it will be very close to 0 in low-density regions

A great idea is to use $\log p(x)$ instead of $p(x)$:

1. $\nabla \log p(x) = \nabla(\log f(x) - \log Z) = \nabla \log f(x)$, therefore we could avoided computing Z
2. $\log$ will make a fuction with numerical stability
3. $\nabla \log p(x)$ represent the same direction of $\nabla p(x)$

补充：

要明白上面为什么反复提到一个Z，需要清楚一个常用的技巧：如果我们希望基于函数$f(x)$（满足非负），构造一个概率密度函数，常见的做法是令$p(x) = f(x)/Z$，其中$Z = \int f(u)du$，即所除以的归一化常数是f在整个空间上的积分。

而上面说直接使用$p(x)$的缺点就是$p(x)$也可以视作某个函数构造出的概率密度函数，在处理它的时候不可避免的要求某个复杂函数在整个高维空间上的积分，这通常是无法计算的，而对数化之后可以通过常数的梯度为0，来避免计算这个复杂积分，让其变得可以处理。

End of prerequisite

usually, we use the gradient $\log p(x)$ as the score function: $s(x) = \nabla_x \log p(x)$

now the question is should we use the score to go from a point easy to sample to highest density regions

the answer is we shouldn't only go to the highest density regions

- if we just follow the socre, we will get to the highest density regions
- but there are also some points landed by the valid data from p_{data} with lower density
- we need to introduce some diversity

the way is called **Langevin Sampling**:

$x_t = x_{t-1} + \frac{\alpha}{2}\nabla_x \log p_{data}(x_{t-1}) + \sqrt{\alpha}\epsilon_t$

1. it will not only allow u go to the higher density regions but also allow u to make a bit exploration

到这里为止我有一个问题：上面一直在讨论，我们希望找到空间中$p(x)$较大的一些区域，我的问题是为什么$p(x)$较大的区域会更可能包含data set中的样本$x_i \sim p_{data}(x)$

实际上，上面讨论的$p(x)$可以直接理解为$p_{data}(x)$，即真实数据的目标分布，而**两个体积相同的小领域中，密度较大的领域更加容易出现真实数据**

而**Langevin Sampling**赋予了我们从某个位置出发，在假设知道真实的目标分布密度函数的梯度时，如何模仿真实的分布进行采样：

1. $\frac{\alpha}{2}s(x_{t-1})$ 使得我们的位置向局部密度增大的方向移动
2. $\sqrt{\alpha}\epsilon_k$ 是随机扩散，它和前者配合，会使得我们的采样在时间步t增长的角度看来，样本的长期分布符合目标分布p_{data}

总的来说问题被转化为了如何学习一个合理的$s_\theta(x)$来拟合$\nabla \log p_{data}(x)$

Score estimation

the ideal score estimation is to use this loss function:

$L_{SM} = E[||s_\theta(x) - \nabla_x \log p_{data}(x)||^2]$

however $\nabla_x \log p_{data}(x)$ is what we don't know

there exist many ways to avoid the distribution p_data and estimate the score

1. Implicit Score Matching $L_{ISM} = E_x[\frac{1}{2}||s_\theta(x)||^2 + \nabla_x s_\theta(x)]$
2. Sliced Score Matching $L_{SSM} = E_{x_0,t,\epsilon,v}[2v^T\nabla_x s_\theta(x)v + |v^T s_\theta(x)|^2]$

now we will look into a common way used today, which is called denoising score match

let's start from Gaussian Distribution:

for a 1-d Gaussian $x \sim N(\mu, \sigma^2)$:

Prbability distribution function:

$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp(-\frac{(x-\mu)^2}{2\sigma^2})$

we can easily get it's actual socre:

$s(x) = -\frac{x-\mu}{\sigma^2}$

why this formula make sense?

1. a typical Gaussian Distribution has the high density on the mean and lower density when we go away
2. easy to see that any sampled x will be closer to μ by added the $s(x)$ so it makes sense

compute the score of Gaussian distribution is easy but it's difficult for a not general distribution

now, we will consider computing the score of some samples form the Gaussian distribution but they are added some noise

Compute the socre of the Gaussian distribution with noise

$$\tilde{x} = x + \sigma\epsilon, \epsilon \sim N(0, I)$$

1. ϵ is the noise from Gaussian distribution
2. σ is a scale factor to scale the noise
3. x is the sample from some typical distribution

~~x is from Gaussian distribution,~~$\tilde{x}$ follows:

note: $\tilde{x}$ has a known distribution which mainly depends on known sample x and σ set by us, and it dosen't depend on the distribution p_{data}

$$q_\sigma(\tilde{x}|x) = N(x, \sigma^2 I)$$

and it's score function:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{\tilde{x}-x}{\sigma^2}$$

1. for the Gaussian distribution, the random varible $\tilde{x}$ by x adding noise has a mean vlaue of x

now let's conisder how to go back the start point about computing the score of p_{data}

the key is to take the p_{data} distribution and noise it

define $q_\sigma(\tilde{x}) = \int q_\sigma(\tilde{x}|x)p_{data}(x)dx$

1. $q_\sigma(\tilde{x}|x)$ is called perturbation kernel to discrible how you add the noise on each sample
2. $p_{data}(x)$ means each sample in the origial distribution, that is to say we repeat the procedure of adding noise for all elements in p_{data}
3. another denote is $q_\sigma(\tilde{x}) = q(\tilde{x}, x)$
4. in practice, we noly have a few data of the p_{data}, what we will do is to noise each samples i.e., $\tilde{x}_i = x_i + \sigma\epsilon$, if $q_\sigma(\tilde{x}|x) = N(x, \sigma^2 I)$
5. σ will control how much noise we add, small σ means a little bit noise and bigger sigma means more noise to make sample spread out and takes much more space.

let's try to use the definition to compte the true score

now we try to optimize the score matching objectives, which is like:

$$L_{SM} = E_{\tilde{x}}[||s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x})||^2]$$

some work has proved that it's actually equal to optimize:

$L_{DSM} = E_{x,\tilde{x}}[||s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x)||^2]$

the derivation about it is use $||a-b||^2 = ||a||^2 - 2 < a, b > + ||b||^2$, and the key is to prove the term about $< a, b >$ is the same

this is a good news! because the perturbation kernel is tractable, like $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}|x) = -\frac{\tilde{x}-x}{\sigma^2}$

actually the loss tractable above is called DSM(denoising score matching), denote L_{DSM}

Limitations with vanilla DSM

a idea is that we can use denoising score matching with a small sigma so the q_{sigma} will very close to the original distribution p_{data}

however if we use a very small σ, we only have a poor estimation of the score in low-density regions

why does this happen?

we want to optimize $L_{SM} = E_{\tilde{x}}[||s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x})||^2]$

what that means is that we want to optimize on the loss that is around the distance between the predicted score and the score of the noise distribution q, which is equal to:

$L = \int ||\Delta_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x})||^2 q_\sigma(\tilde{x}) d\tilde{x}$

1. we weight our loss function with respect to how likely our points $\tilde{x}$ are to appear
2. if $\tilde{x}$ is not likely to appear, then the factor it is off from the true value is not something that is going to be materially noticeable in the loss, which makes the model will not really be incentivized

说明：

1. 原始的损失形式是期望，上面L的写法就是将期望按照积分的形式展开
2. 这个损失的含义是，按照$\tilde{x}$的真实出现频率，对预测的误差求平均，在某一个小区域内，如果出现的频率大，也就是$q_\sigma(\tilde{x})$大，对应区域求得的$||\Delta_\theta(\tilde{x}) - \nabla \log q_\sigma(\tilde{x})||^2$对总体损失函数的值就比较大
3. 进而，会引发的问题是，假设实际上目前模型预测效果很差，损失函数**本应该较大**，但是由于目前样本形成的$\tilde{x}$所出现的区域中，模型预测的表现效果比较好（假设这一部分只占了总体区域的百分之1），那么损失函数会较小，看起来模型已经优化好了，但是实际上在其余百分之99的区域，只要取了对应的$\tilde{x}$损失函数就会非常大，模型表现仍然很差
4. 训练样本集中在真实图片附近；但生成时，我们往往从简单的高斯噪声开始。这样的初始点可能远离小噪声数据分布的主要区域。此时，我们希望 score 能提供可靠的方向，把样本逐渐带向数据区域。可是模型在这些起始位置缺少训练，预测的方向就可能不准确。

this is a trade-off: small σ means the distribution q_σ (whose score function we use) is closer to true distribution q_{data} which we want, but the trained model have a poor ability to predict an accurate socre function in some general position (like the point we start from, when we want to make a img generation); big σ means q_σ is more different from q_{data}, but the score predicted by the model of q_σ in some general position is more accurate

Score matching with varying noise via NCSN

to overcome the problem in last section, a great idea is:

we are not goint to only rely on one kind of noisy distribution, and we're going to actually rely on all of the sigma from a lower value to a higher value

therefore we can release a effect that in the very beginning we know roughly where to go and the more we get closer to the high density region of the true distribution we know more and more about the details

the method is called NCSN = Noise Conditional Score Network:

1. we estimate score $s_\theta(x, \sigma_i)$ at different noise level $\sigma_1 < ... < \sigma_T$
2. we use the loss: $L_{NCSN} = \sum_{i=1}^{L} \lambda(i) E_{x \sim q_{\sigma_i}}[||s_\theta(x, \sigma_i) - \nabla_x \log q_{\sigma_i}(x)||_2^2]$

NCSN could predict an accurate score under different noise level and if we wanna to leverge these scores to sample step by step until get a good result x_t, we need to use ALD = Annealed Langevin Dynamics (another sampling method)

ALD:

1. sample noise $x_T \sim N(0, \sigma_T^2 I)$, $\sigma_T > ... > \sigma_1$
2. For each σ_i, perform iterative update for K steps:

- $x \leftarrow x + \frac{\alpha_i}{2} s_\theta(x, \sigma_i) + \sqrt{\alpha_i}\epsilon$

3. obtain final image x_0

Parallel between noise (DDPM) and score (NCSN)

a natural question is how dose the score relate to the noise prediction task

we know DDPM is about learn how to denoise under a permise that we have added the noise

1. we add the noise by $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$
2. by the formula above, $q(x_t|x_0) = N(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I)$

the actul reality is that: the score of the distribution $q(x_t|x_0)$ is equal to the noise we added over some coefficient:

$$\nabla \log q(x_t|x_0) = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}$$

1. actually these two methods are two different ways of noising our sample (one is use weighted average between clean img and noise, and another one is just use sigma for clean img)
2. this is why DDPM is seen as variance presevring (the variance of x_t in DDPM is roughly equal to 1), but NCSN is going to be mentioned as being variants exploding (because we're going to add more and more noise (using σ), which is not bounded by 1)

Continous formula

Up to now, noising img by us uses a discrete set of noise increments

So one idea is to acutally consider continous evolution instead of discrete one

(the reason why we wanna to do this is beacuase we have a filed of math which is differential equations that have a bunch of convenient toolkits to solve these kinds of equations)

补充：

1. continous formula 连续时间形式，可以理解为把按照$t = 1, 2, 3, ..., T$的离散时间步加噪，推广为“图像随着连续时间t不断受到微小的随机扰动”
2. 它为 DDPM 和基于 score 的生成方法提供了统一描述，这个统一框架通常用随机微分方程SDE表示。
3. 这样做的优点：

- 统一不同方法。 DDPM 的加噪过程和 NCSN 使用的多噪声尺度，可以放进同一个 SDE 框架。
- 更灵活地设计采样算法。 给定连续变化规律后，可以选择不同的数值求解器、时间网格和步长。
- 更方便分析生成过程。 例如研究概率分布怎样随时间变化，以及它与确定性 ODE 的关系。

Wiener process

let's try continous equivalent of adding increments of Gaussian noise

Wiener process is a type of stochasitc process which is a family of random varibles that are indexed by some index t

It needs to follow some special properties:

1. $W_0 = 0$
2. $W_t - W_s \sim N(0, (t-s)I)$
3. independent increments

补充：

1. Wiener process是维纳过程（也称标准布朗运动），可以记做$\{W_t\}_{t \geq 0}$，有两种理解角度：

- 固定一个时刻t，W_t是一个随机变量
- 观察一次实验的全部时刻，得到一条随机轨迹$t -> W_t$
- (可以想象：运行一次程序，生成一条随机变化的曲线；重新运行一次，又生成另一条曲线。维纳过程描述的是这些随机曲线所遵循的概率规律。)

2. 关于维纳过程要遵循的三条性质：

- 从零开始，表示最初没有积累任何随机扰动
- 任意时间区间上是高斯分布
- 第三条指的是，不相交时间区间上的增量相互独立，即$W_1 - W_0, W_2 - W_1$是相互独立的，但是注意W_0, W_1, W_2并非独立的（考虑$W_2 = W_1 + (W_2 - W_1)$，并结合性质2就可以得出这个结论）
- 实际上还有第四条性质，即轨迹连续，时间连续，轨迹不能出现跳跃

now let's derive the continous formula of DDPM:

1. starting from decrete formula:

- $x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$
- $x_t - x_{t-1} = (\sqrt{1-\beta_t} - 1)x_{t-1} + \sqrt{beta_t}\epsilon$

2. define a quantity: $\beta = \beta(t)dt$

- $\beta(t)$ is the rate of noise at time t
- dt is an increment of time

3. to get a continous formula

- $x_t - x_{t-1} = (\sqrt{1-\beta(t)dt} - 1)x_{t-1} + \sqrt{\beta(t)dt}\epsilon$
- **tend t to 0 and consider a continous evolution**, $x_t - x_{t-1}$ is going to be a differential evolution dx_t
- when dt tends to 0, we have Taylor expansion formulas: $dx \sim (1 - \frac{1}{2}\beta(t)dt - 1)x + \sqrt{beta(t)}dW$ where the $dW = \sqrt{dt}\epsilon$ because dW follows a normal distribution of variance dtI
- finally we get: $dx = -\frac{1}{2}\beta x dt + \sqrt{\beta}dW$

this equation is a formula of Stochastic Differential Equation(SDE)

$$dx = f(x,t)dt + g(t)dW$$

1. $f(x,t)$ is called drift and it's deterministic
2. $g(t)$ is called diffusion which is used to control the strength of randomness term (like dW)

just see how the SDE is used:

1. suppose we have a start point x and only have the drift term:

- we will have a determistic evolution using $x < -x + dx$, it finally will be a determistic route

2. if we consider the whole terms, a final step will include a determistic step and a stochastic step

therefore in DDPM's continous formula:

- $f(x,t) = -\frac{1}{2}\beta(t)x$
- $g(t) = \sqrt{\beta(t)}$

NCSN's continous formula:

- $f(x,t) = 0$
- $g(t) = \sqrt{\frac{d(\sigma^2(t))}{dt}}$

trianing and inference

now we know how to add noise with a continous formlua, but we care more about is how to denoise

traning obejective is alos predict the socre:

$$L_{DSM} = E_{t,x_0,x_t}[\lambda_t||s_\theta(x_t,t) - \nabla_{x_t}\log p(x_t|x_0)||^2]$$

training process:

1. sample clean img $x_0 \sim p_{data}$, noise $\epsilon \sim N(0,I)$ and time step $t \sim \mathcal{U}(0,T)$
2. noised img $x_t = \alpha_t x_0 + \sigma_t \epsilon$
3. use x_t and t to predict $-\frac{\epsilon}{\sigma}$ via $s_\theta(x_t,t)$
4. compute loss $L = \lambda_t||s_\theta(x_t,t) + \frac{\epsilon}{\sigma_t}||^2$ and backpropagate through s_θ

说明：

- 这一部分是重复回顾了一下我们之前学过的L_{DSM}，训练的网络用于预测的是一个socre项

- 结合后面的推理理解，因为推理的时候要用到这个score项

for inference, we need the reverse of the original continous formula above, which represents denoising process

$$dx = [f(x,t) - g(t)^2 \nabla_x \log(p(x))]dt + g(t)d\langle W|$$

1. this is the advantages of continous formula: the forward process described as SDE can be reversed through a reverse SDE
2. the socre term $\nabla_x \log(p(x))$ has appereared in the reverse SDE
3. $d\bar{W}$ means that it's different from dW

inference:

1. sample noise $x_T \sim N(0, \sigma_T^2 I)$
2. use Euler-Maruyama to go through reverse SDE: $x_{t_{i-1}} = x_{t_i} + [f(x_{t_i}, t_i) - g(t_i)^2 s_\theta(x_{t_i}, t_i)]\Delta t + g(t_i) + \sqrt{\Delta t}\epsilon_i$
3. Obtain final img x_0

说明：上面的i指的是可以在求解数值解的时候设置的迭代次数（Euler-Maruyama本身就是一种数值求解法）

单词对照

partial derivation 偏导

integral 不可缺少的

redundant 多余的

perturbation kernel 扰动核

trivial 琐碎细节的

incentivized 激励

discrete 离散的

increment 增量